



CONSENSUS

Termination: *all P_c decides exactly 1 value v_i*

Agreement: *all P_c decides the same v_i*

Validity: *v_i was proposed by some P_i*

REPLAY



Impossibility Result for process crash failures

FLP'85 Impossibility Result

**There is no deterministic
1-resilient fail-stop Consensus solution
in an asynchronous system.**

REPLAY

FLP'85 = Fischer-Lynch-Paterson 1985 r. → [\[1\]](#)



Workarounds

- (1) Synchrony – synchronous (quasi-synchronous) system model
- (2) Weaker fault model – e.g. in the *initially dead-processes* model, weaker than *fail-stop* model, consensus and election are deterministically achievable
- (3) Sacrifice liveness for safety = relaxation of the termination condition, e.g.
- (3') randomization:

Termination: every P_c eventually decides with probability 1

- (3'') or Paxos



PAXOS (LAMPORT [2])

Model assumptions (initial):

- $\mathbb{S}^{\text{Async}}\{\emptyset\}$
- fail-stop process failures, $f < \frac{N}{2}$, i.e. at least $M = \left\lceil \frac{N+1}{2} \right\rceil$ processes are correct
- reliable communication channels

General idea:

- a proposal is chosen as a decision if *accepted* by majority of processes
- each proposal has a unique number pn_i
- only the proposal with the highest number pn_i wins (i.e. is *accepted*)
- a proposal once *accepted* will remain so
- $\mathbb{S}^{\text{Async}}\{\text{uniqID}\}$: $pn_i = i$



Paxos basic

1. P_i broadcasts PREPARE request with pn_i to all (including itself ¹⁾)
2. any receiver P_j responds to PREPARE with:
 - OK – a promise to never again accept a proposal numbered less than pn_i if pn_i is greater than that of any PREPARE to which it has already responded or
 - ACCEPTED with the proposal v_k that P_j has already accepted with pn_k (pn_k may be less than pn_i)
3. if P_i receives a response (OK or ACCEPTED) from majority M of processes, it:
 - sets $v_i = v_k$ with the highest pn_k from among all ACCEPTED responses, if any
 - and, in that case, also sets $pn_i = pn_k$
 - then broadcasts PROPOSE with v_i and pn_i to all
4. when P_j receives PROPOSE with v_i and pn_i it accepts the proposal v_i unless it has already responded to a PREPARE request with higher pn_k
5. if P_i has accepted a value v_i , it broadcasts an ACCEPTED message with v_i and pn_i
6. when P_i receives ACCEPTED with the same v_j from M processes, it decides v_j

¹⁾ In reality, P_i does not need to send messages to itself, but can simply include the message it broadcasts directly in the collection of messages it receives later on



Model assumptions (cont.)

- omission failures in communication channels:
 - P_i sends broadcasts using RBcast
- fail-recovery process failure model:
 - $\mathbb{S}^{\text{Async}}\{\text{stable storage}\}$: P_i has to remember, in its stable storage, the $\mathbf{v}_k$ it has accepted (step 5.)
or
the highest $\mathbf{pn}_k$ it has agreed to accept (step 2.)



Termination in $\mathcal{S}^{\text{Async}}\{\emptyset\}$

- Paxos = relinquishment of the Termination condition
- if the process winning step 3, and about to propose, fails $\rightarrow$ livelock
- if we allow processes to restart (to retry their would-be proposals) with incremented pn_i , it may never come to an end (no absolute winner ever in step 3) $\rightarrow$ still livelock
- this, however, requires $\mathcal{S}^{\text{Async}}\{\text{Election}\} \rightarrow$ Paxos extended [3], Raft [4]
- then, to guarantee progress in, a distinguished process can be elected as the only one issuing PROPOSE (in step 3)
- for multiple (concurrent) instances of Paxos $\rightarrow$ Paxos multi [2]



Impossibility Result for termination in $\mathcal{S}^{\text{Async}}\{\emptyset\}$

**There is no 1–resilient fail-stop
solution to Consensus
that always terminates.**

→  [5]



PROBABILISTIC SOLUTIONS

Monte Carlo

A probabilistic algorithm is Monte Carlo if:

- it always terminates
- the probability of getting a correct terminal configuration is greater than zero

Las Vegas

A probabilistic algorithm is Las Vegas if:

- it terminates with probability greater than zero
- all terminal configurations are correct



Impossibility Result for probabilistic solutions

**There is no 1–resilient fail-stop
Monte Carlo solution to Consensus.**

→  [5]



Impossibility Result for probabilistic solutions (2)

**There is no f -resilient fail-stop
Las Vegas solution to Consensus
for $f \geq \frac{N}{2}$.**

→  [5]

If $f < \frac{N}{2}$, then an f -crash resilient Las Vegas solution for consensus does exist.



LAS VEGAS BINARY CONSENSUS ALGORITHM (BRACHA-TOUEG [5])

Model:

- $\mathcal{S}^{\text{Async}}\{\emptyset\}$
- fail-stop process failures, $f < \frac{N}{2}$
- reliable communication channels

General idea:

- initially (at the start of round 0) each process randomly chooses a value 0 or 1 as $\mathbf{v}_i$
- with *weight* $\mathbf{w}_i=1$
- an undecided process determines a new value $\mathbf{v}_i$ and weight $\mathbf{w}_i$, based on the first $N-f$ messages it receives in a given round
- the weight approximates the number of processes that voted $\mathbf{v}=\mathbf{v}_i$ in the previous round



Round $k \geq 0$:

- P_i broadcasts $\langle k, v_i, w_i \rangle$ to all (including itself²)
- P_i awaits $\langle k, v_j, w_j \rangle$ from $N-f$ processes
- if $w_j > \frac{N}{2}$ for an incoming message $\langle k, v_j, w_j \rangle$, then $v_i = v_j$
- if $w_j > \frac{N}{2}$ for more than f incoming messages, then P_i decides v_i
and broadcasts $\langle k+1, v_i, N-f \rangle$ and $\langle k+2, v_i, N-f \rangle$, and then terminates
- if $w_j \leq \frac{N}{2}$, then $v_i = \text{majority}(v_j)$ and $w_i = \text{number of incoming votes}$ $v = \text{majority}(v_j)$
(say, default = 0, as usual)
- when a process decides, all other correct processes are guaranteed also to decide within 2 rounds

²) *Again, in reality, P_i does not need to send messages to itself, but can virtually include the message it broadcasts in the collection of messages it receives in a round (calculating w_i appropriately).*



Example:

$$N=3; f=1$$

$$\begin{matrix} v_1=1 \\ w_1=1 \end{matrix} \quad \textcircled{P_1}$$

$$\begin{matrix} v_2=1 \\ w_2=1 \end{matrix} \quad \textcircled{P_2}$$

$$\begin{matrix} v_3=0 \\ w_3=1 \end{matrix} \quad \textcircled{P_3}$$

Each round a correct process requires 2 incoming messages, and 2 votes $\mathbf{v}$ with weight 2 to decide $\mathbf{v}$.

$$>f$$

$$>\frac{N}{2}$$

$$N-f$$

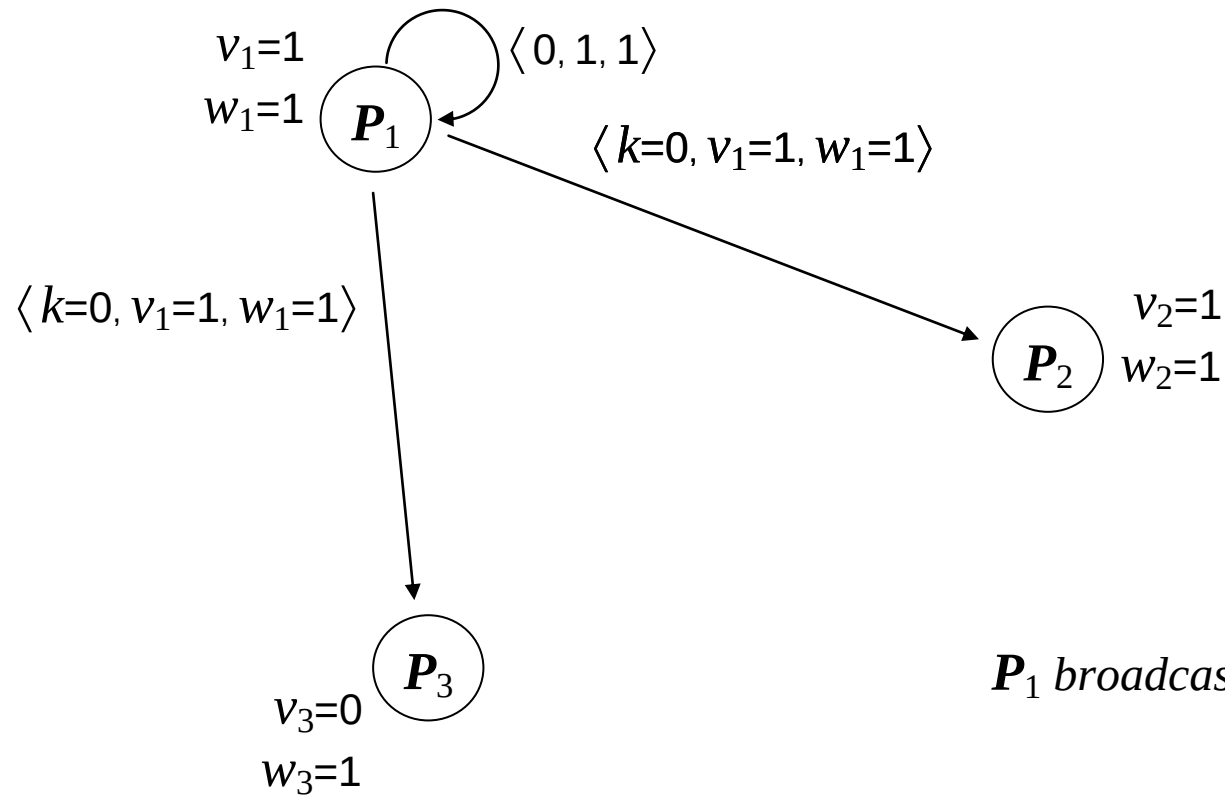
Initially, P_1 and P_2 randomly choose the value 1 and P_3 the value 0.



Example (round $k=0$):

$N=3; f=1$

Each round a correct process requires 2 incoming messages, and 2 votes $\mathbf{v}$ with weight 2 to decide $\mathbf{v}$.



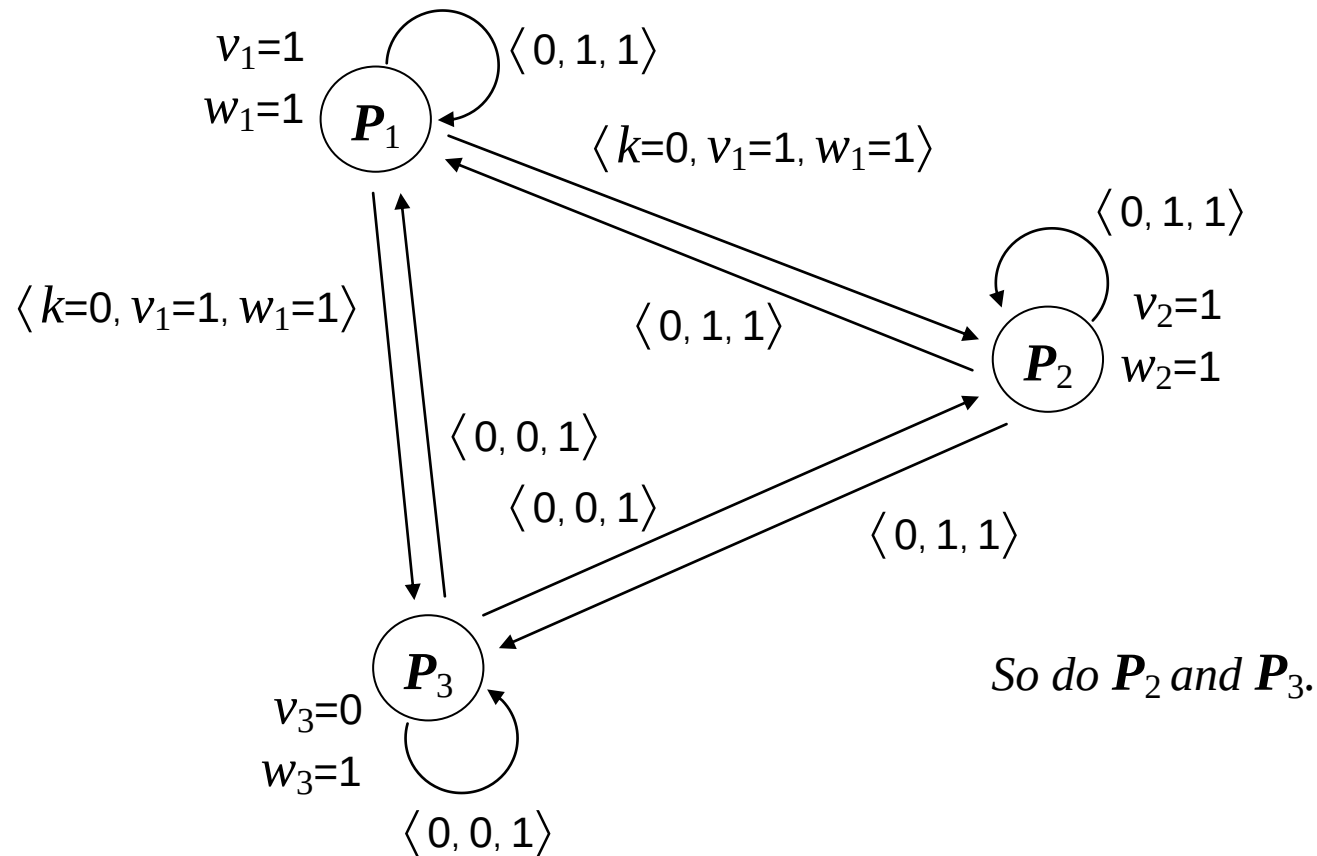
P_1 broadcasts its message.



Example (round $k=0$):

$N=3; f=1$

Each round a correct process requires 2 incoming messages, and 2 votes $\mathbf{v}$ with weight 2 to decide $\mathbf{v}$.

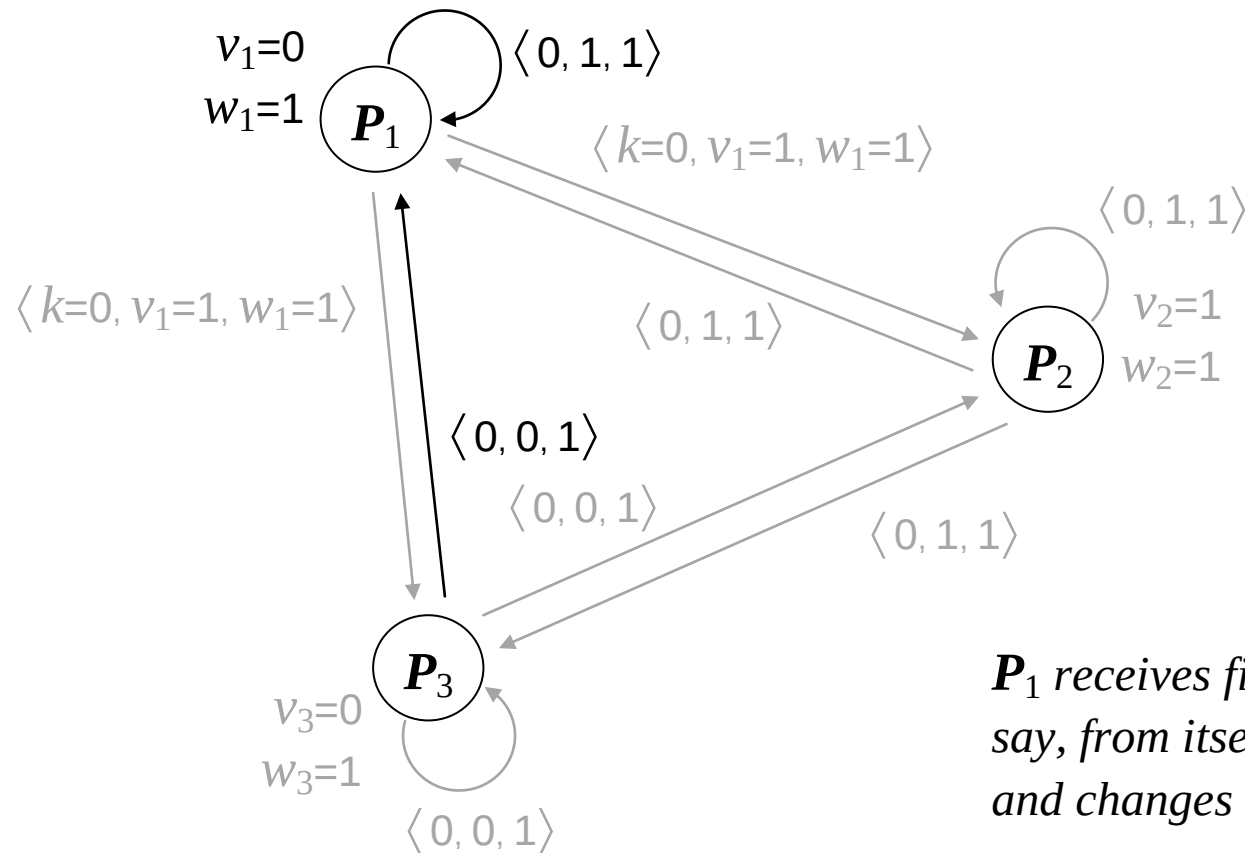




Example (round $k=0$):

$N=3; f=1$

Each round a correct process requires 2 incoming messages, and 2 votes $\mathbf{v}$ with weight 2 to decide $\mathbf{v}$.



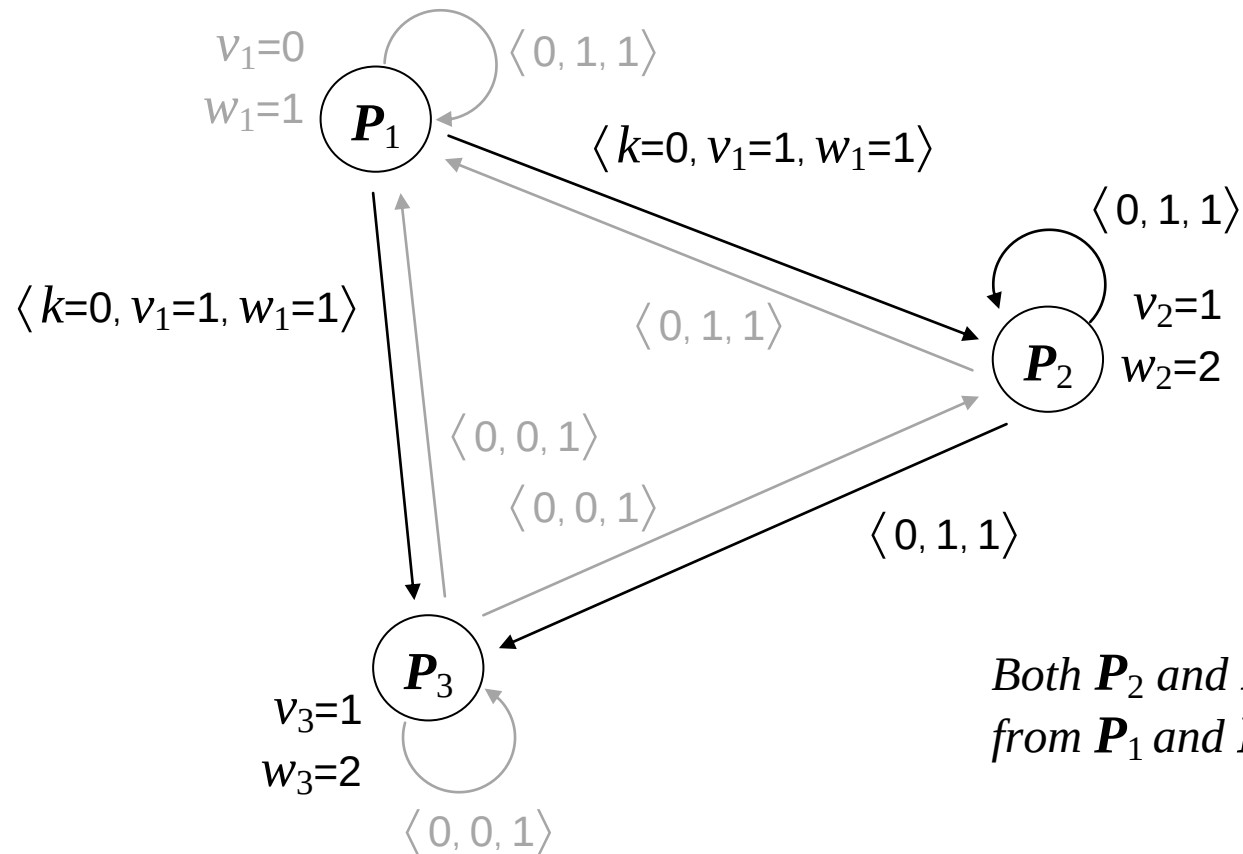
P_1 receives first 2 incoming messages, say, from itself — P_1 and from P_3 , and changes its value $\mathbf{v}_1$ to **majority**($\mathbf{v}_j$).



Example (round $k=0$):

$N=3; f=1$

Each round a correct process requires 2 incoming messages, and 2 votes v with weight 2 to decide v .



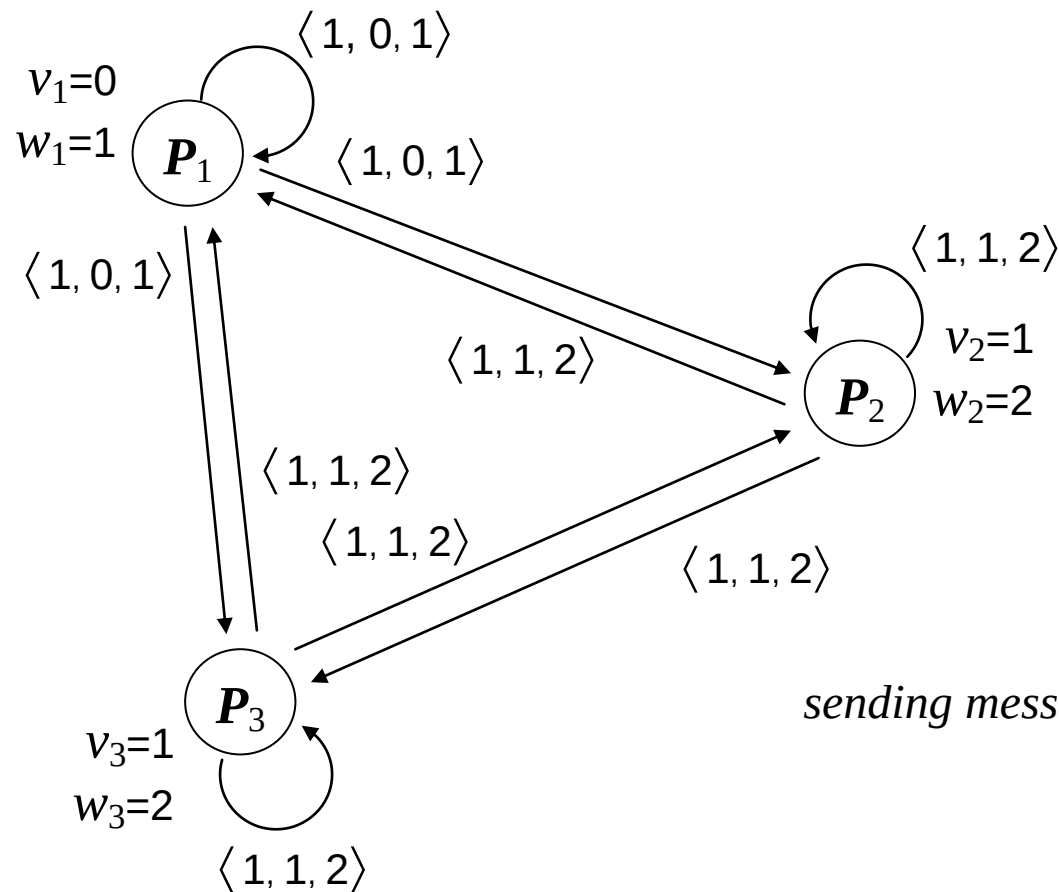
Both P_2 and P_3 receive messages from P_1 and P_2 .



Example (round $k=1$):

$N=3; f=1$

Each round a correct process requires 2 incoming messages, and 2 votes v with weight 2 to decide v .

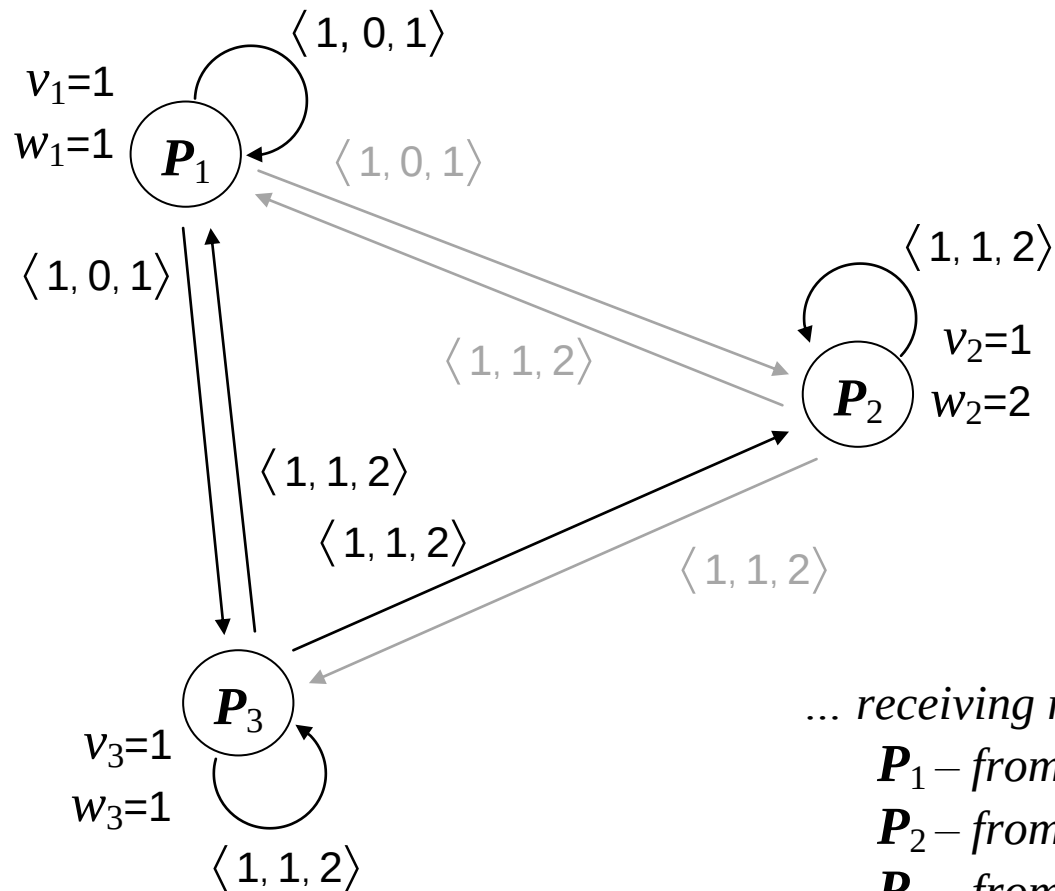




Example (round $k=1$):

$N=3; f=1$

Each round a correct process requires 2 incoming messages, and 2 votes v with weight 2 to decide v .



... receiving messages:

P_1 – from P_1 and P_3 ,

P_2 – from P_2 and P_3 ,

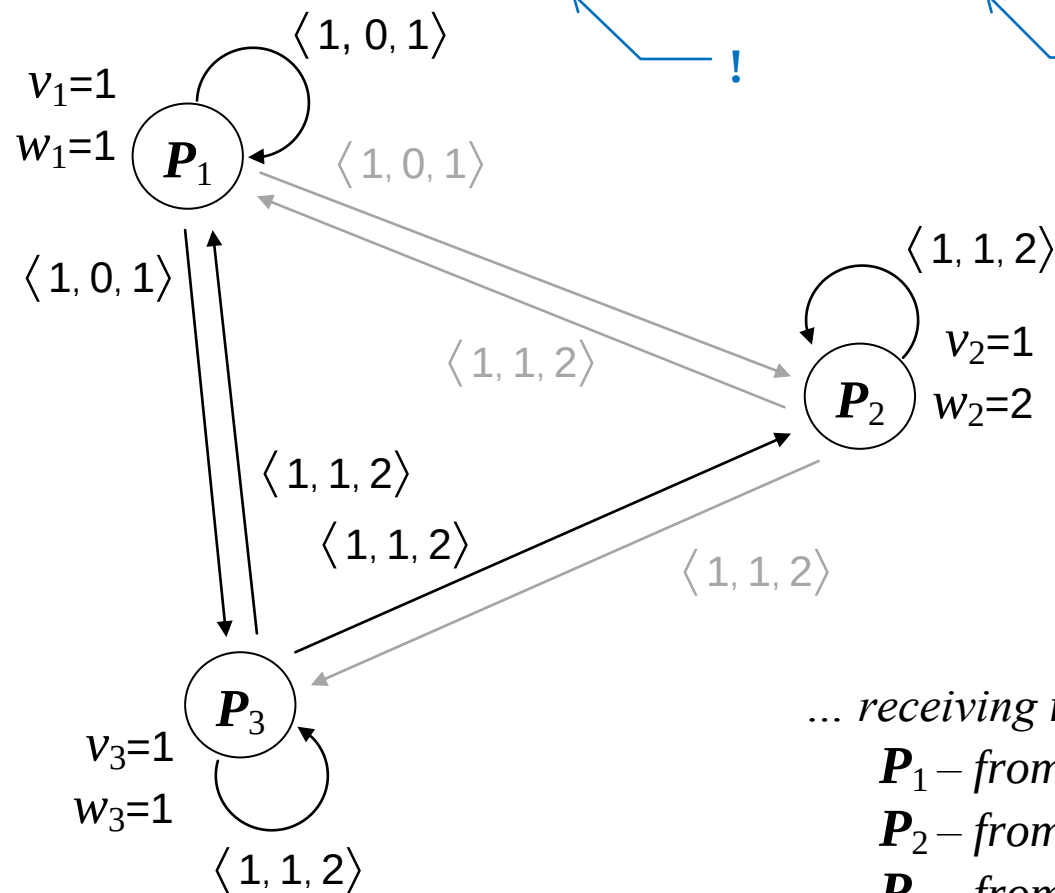
P_3 – from P_1 and P_3



Example (round $k=1$):

$N=3; f=1$

Each round a correct process requires 2 incoming messages, and 2 votes v with weight 2 to decide v .



... receiving messages:

P_1 – from P_1 and P_3 ,

P_2 – from P_2 and $P_3 \rightarrow$ *decides 1*,

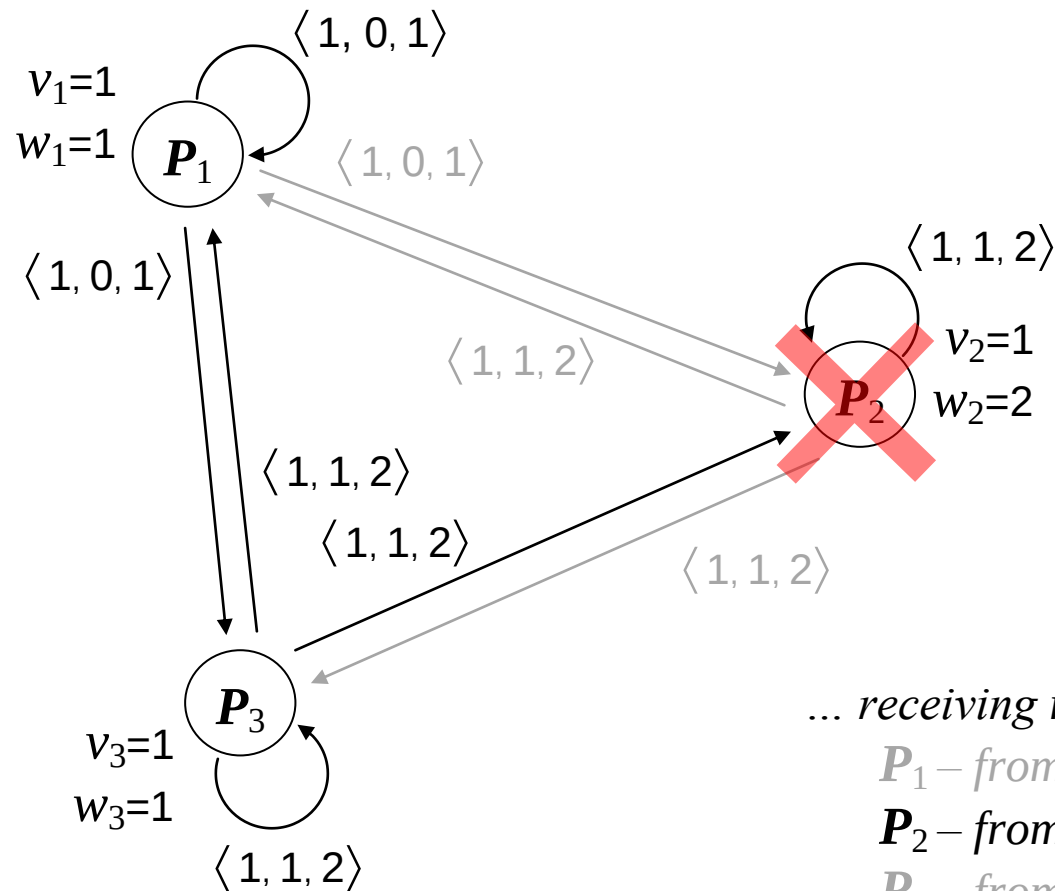
P_3 – from P_1 and P_3



Example (round $k=1$):

$N=3; f=1$

Each round a correct process requires 2 incoming messages, and 2 votes v with weight 2 to decide v .



... receiving messages:

P_1 – from P_1 and P_3 ,

P_2 – from P_2 and $P_3 \rightarrow$ **and crashes!**

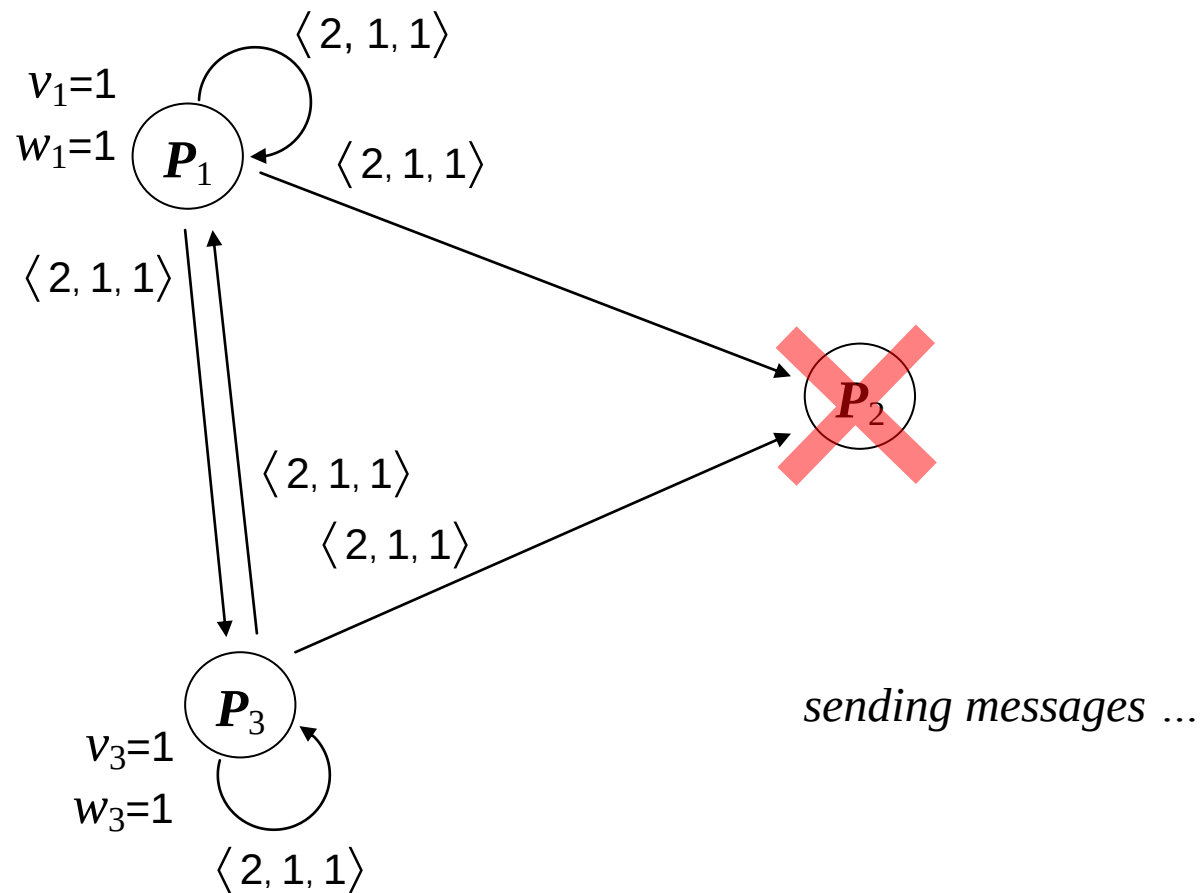
P_3 – from P_1 and P_3



Example (round $k=2$):

$N=3; f=1$

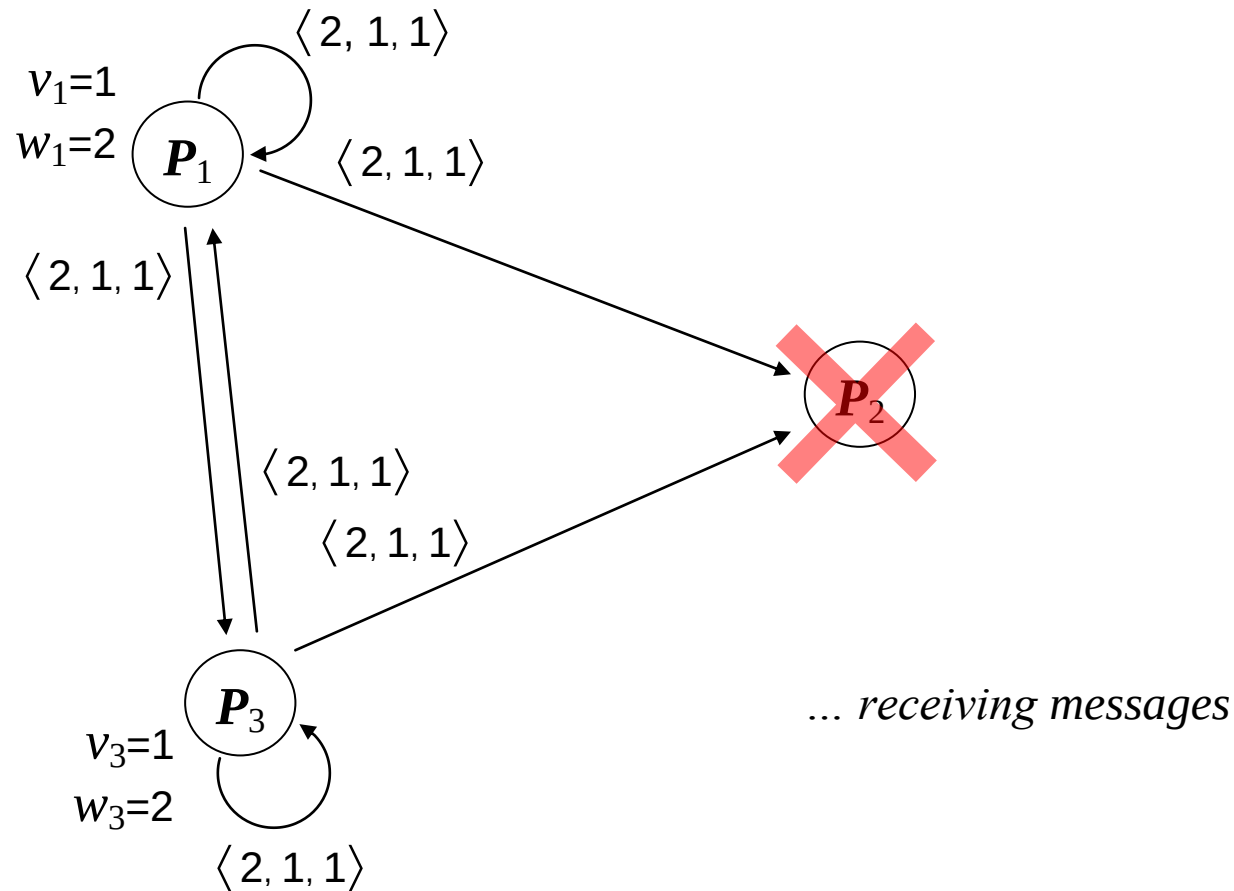
Each round a correct process requires 2 incoming messages, and 2 votes $\mathbf{v}$ with weight 2 to decide $\mathbf{v}$.



Example (round $k=2$):

$N=3; f=1$

Each round a correct process requires 2 incoming messages, and 2 votes $\mathbf{v}$ with weight 2 to decide $\mathbf{v}$.

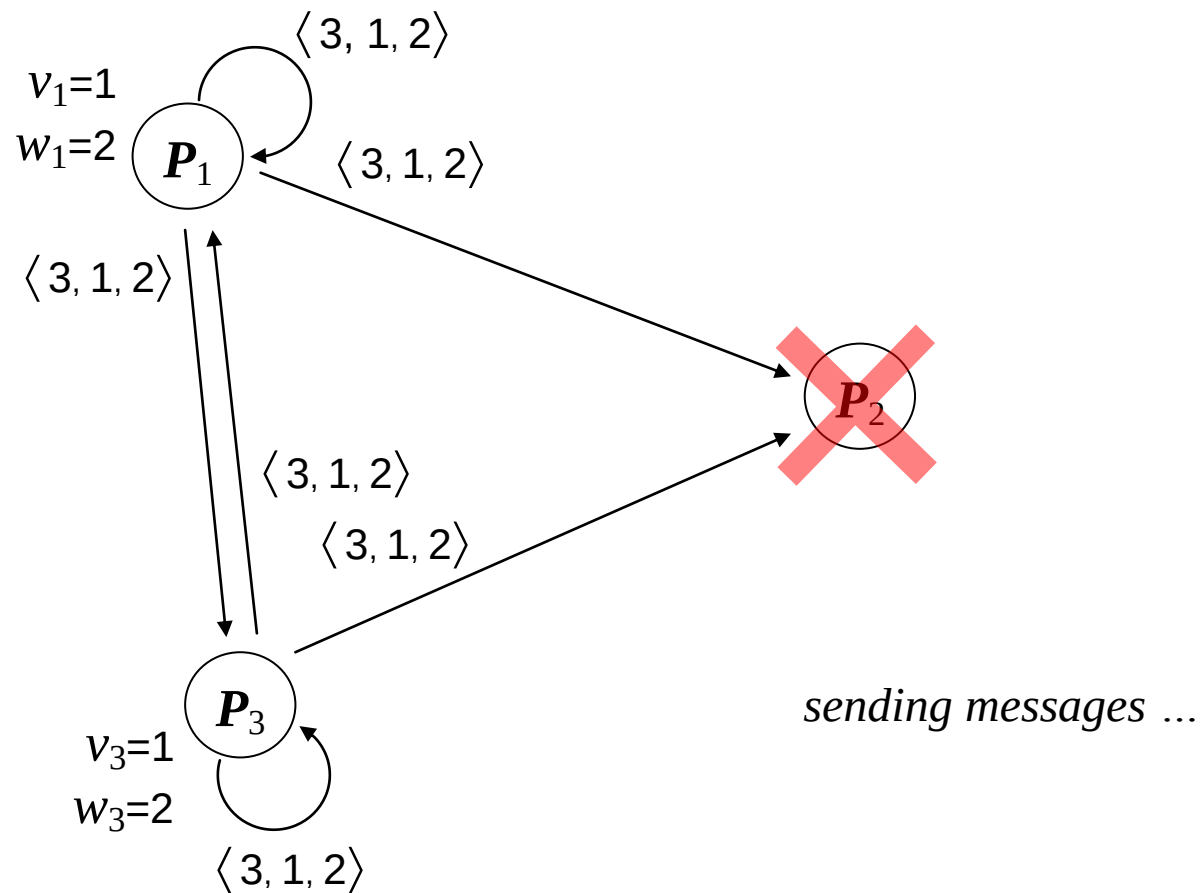




Example (round $k=3$):

$N=3; f=1$

Each round a correct process requires 2 incoming messages, and 2 votes v with weight 2 to decide v .

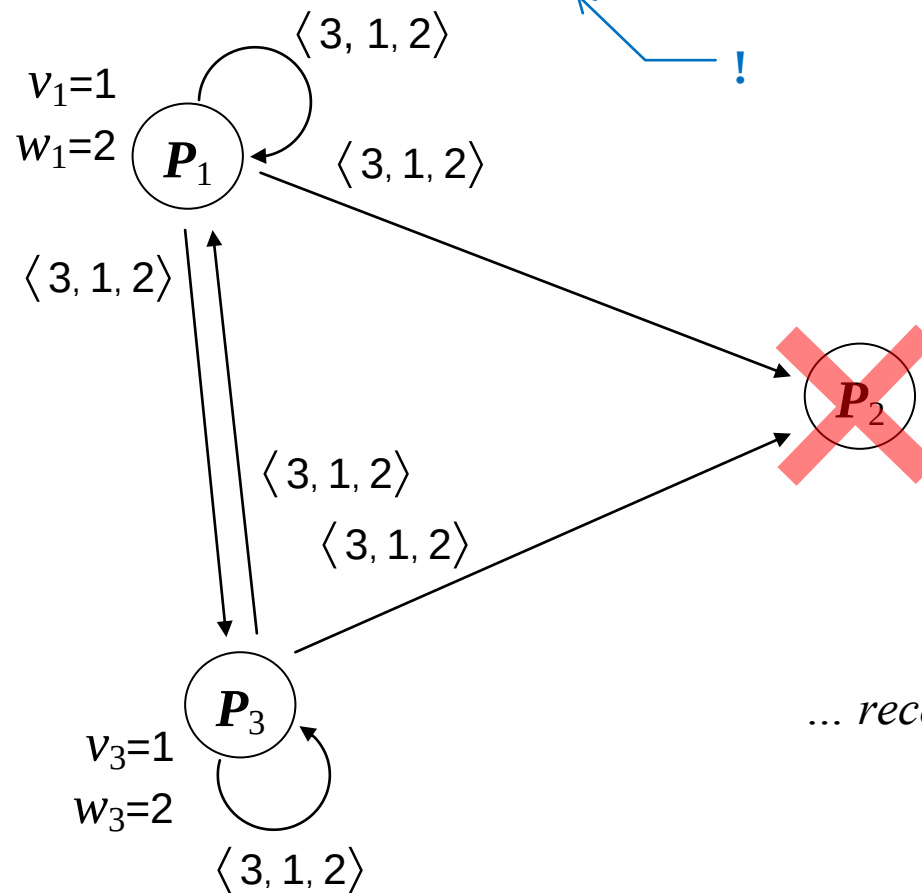




Example (round $k=3$):

$N=3; f=1$

Each round a correct process requires 2 incoming messages, and 2 votes $\mathbf{v}$ with weight 2 to decide $\mathbf{v}$.



... receiving messages $\rightarrow$ decisions



Consider (as your homework) other variants of this example:

1. What if P_2 terminates before crashing?
2. Follow the algorithm's execution with different messages received in particular rounds.



Correctness:

Note that if P_i waited for more than $N-f$ messages in a round k , it would lead to a starvation, in case f processes have crashed.

Note also that if P_i receives messages $\langle k, v_x, w_x \rangle$ and $\langle k, v_y \neq v_x, w_y \rangle$, then $w_x + w_y \leq N$, so w_x and w_y cannot both be greater than $\frac{N}{2}$.

Note, finally, that since P_i waits for $N-f$ messages $\langle k, v_j, w_j \rangle$ and can decide only if more than f of those messages have their weight greater than $\frac{N}{2}$, it is essential that $N-f > f$, thus $f < \frac{N}{2}$ is required.



Correctness:

Theorem 1:

For $f < \frac{N}{2}$, the Bracha-Toueg consensus algorithm is a Las Vegas algorithm that terminates with probability 1.

Proof:

First we prove that processes cannot decide for different values.



Proof:

First we prove that processes cannot decide for different values.

- Suppose P_i decides v in round k .
- Then in round k , P_i has $v_j=v$ and $w_j > \frac{N}{2}$ for more than f received votes.
- So in round k , each correct process receives at least one $\langle k, v, w \rangle$ with $w > \frac{N}{2}$.
- So in round $k+1$, all correct processes vote v .
- So in round $k+2$, all correct processes vote v with $w=N-f$.
- Hence, at the end of round $k+2$, all correct processes have decided the same value v .



Proof:

- Since the channels are reliable, there is a chance $p > 0$ that in each round k all processes receive the first $N-f$ messages from the same processes.
- After that round k , all correct processes have the same value v .
- After round $k+1$, all correct processes have the same value v with $w=N-f$.
- After round $k+2$, all correct processes have decided value v .
- So, the algorithm terminates with probability 1.





CONSENSUS OF BYZANTINE PROCESSES

Phase-King algorithm (Berman-Garay[6]) for binary consensus

Model assumptions:

- $\mathcal{S}^{\text{Sync}}\{\emptyset\}$
- Byzantine process failures, $f < \frac{N}{4}$ malicious processes
- reliable communication channels

General idea:

- $f+1$ phases (each with two steps)
- rotating coordinator: a distinct process is a leader in each phase (the phase king)



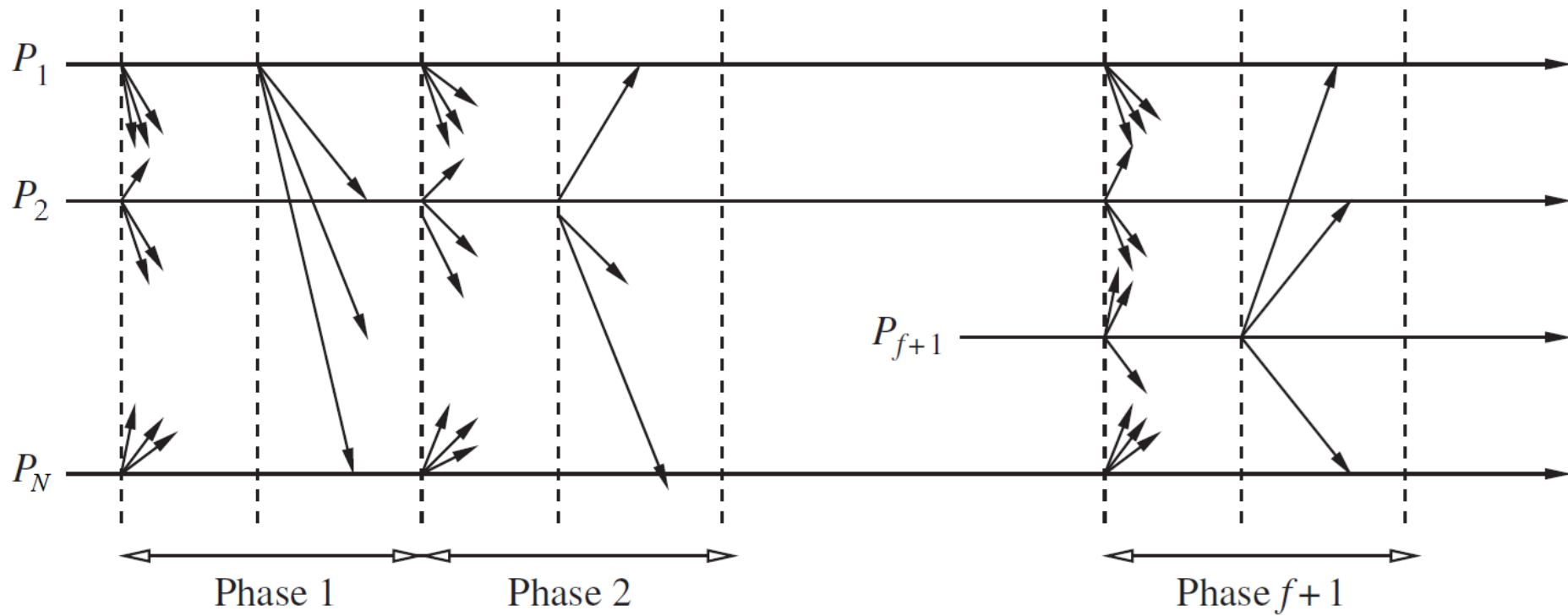
P_i executes the following $f+1$ phases:

```
for phase = 1 to  $f+1$  do
  Execute the following step 1 actions:
    broadcast  $v_i$  to all processes;
    await value  $v_j$  from each process  $P_j$ ;
    majority = majority( $v_1, \dots, v_N$ );
     $w_i$  = number of times that majority occurs;

  Execute the following step 2 actions:
    if  $i == \text{phase}$  then
      broadcast majority to all processes;
    receive tiebreaker from  $P_{\text{phase}}$ 
      (default value if nothing is received);
    if  $w_i \leq N/2 + f$  then  $v_i = \text{tiebreaker}$ ;
    else  $v_i = \text{majority}$ ;
    if  $\text{phase} == f+1$  then decide value  $v_i$ .
```



- polynomial number of messages:





Step 1

```
... Execute the following step 1 actions:  
    broadcast  $v_i$  to all processes;  
    await value  $v_j$  from each process  $P_j$  ;  
    majority = majority( $v_1, \dots, v_N$ );  
     $w_i$  = number of times that majority occurs;
```

- in the first step of each phase, each process broadcasts its estimate of the consensus value to all other processes
- and likewise awaits the values broadcast by others



Step 1

```
... Execute the following step 1 actions:  
    broadcast  $v_i$  to all processes;  
    await value  $v_j$  from each process  $P_j$ ;  
    majority = majority( $v_1, \dots, v_N$ );  
     $w_i$  = number of times that majority occurs;
```

- if any number occurs more than $\frac{N}{2}$ times, then P_i sets its *majority* variable to that consensus value
- and sets w_i to the number of votes received for the majority value
- if no vote occurs more than $\frac{N}{2}$ times
(which may happen when the malicious processes do not respond,
and the correct processes are split among themselves),
then a default value (say, 0, as usual) is used for the majority



Step 2

```
... Execute the following step 2 actions:  
    if i == phase then  
        broadcast majority to all processes;  
    receive tiebreaker from  $P_{\text{phase}}$   
        (default value if nothing is received);  
    if  $w_i \leq N/2 + f$  then  $v_i = \text{tiebreaker}$ ;  
    else  $v_i = \text{majority}$ ;  
    if phase ==  $f+1$  then decide value  $v_i$ .
```

- in the second step of each phase, the phase king initiates processing (the phase king P_{phase} for phase k is the process P_k)
- it broadcasts its majority value `majority`, which will serve the role of a tie-breaker vote for those other processes that would not get their value of w_i more than $\frac{N}{2} + f$



Step 2

```
... Execute the following step 2 actions:  
...  
    receive tiebreaker from  $P_{\text{phase}}$   
        (default value if nothing is received);  
    if  $w_i \leq N/2 + f$  then  $v_i = \text{tiebreaker}$ ;  
    else  $v_i = \text{majority}$ ;  
    if  $\text{phase} == f+1$  then decide value  $v_i$ .
```

- when P_i receives the tie-breaker, it updates its estimate of the decision variable v_i to the value sent by the phase king if its own $w_i \leq \frac{N}{2} + f$
- because among the votes for its own `majority` value, f votes could be bogus
- hence P_i does not have a clear majority of votes (i.e., $> \frac{N}{2}$) from correct processes
- therefore, P_i adopts the value of the phase king



Step 2

```
... Execute the following step 2 actions:  
...  
receive tiebreaker from  $P_{\text{phase}}$   
    (default value if nothing is received);  
if  $w_i \leq N/2 + f$  then  $v_i = \text{tiebreaker}$ ;  
else  $v_i = \text{majority}$ ;  
if phase ==  $f+1$  then decide value  $v_i$ .
```

- however, if $w_i > \frac{N}{2} + f$, then it has received a clear majority of votes from the non-malicious processes
- and hence it updates its estimate of the consensus variable v_i to its own majority value
- irrespective of what tie-breaker value the phase king has sent in this step



At the end of $f+1$ phases

... Execute the following round 2 actions:

...

receive tiebreaker from P_{phase}
(*default value if nothing is received*);

if $w_i \leq N/2 + f$ **then** $v_i = \text{tiebreaker}$;

else $v_i = \text{majority}$;

if $\text{phase} == f+1$ **then** decide value v_i .

- the estimate v_i of all correct processes is the correct consensus value



Correctness

- 1) Among the $f+1$ phases:
 - the phase king of some phase k is non-malicious
 - because there are at most f malicious processes
- 2) As the phase king of that phase k is non-malicious:
 - all non-malicious P_i have the same estimate value v_i at the end of phase k
 - since any two non-malicious processes P_i and P_j can set their estimate v in three ways:
 - (a) both P_i and P_j use their own `majority` values:
 - assume P_i 's `majority` value is x , which implies that P_i 's $w_i > \frac{N}{2} + f$,
 - and of these voters, at least $\frac{N}{2}$ are non-malicious
 - this implies that P_j must also have received at least $\frac{N}{2}$ votes for x ,
 - implying that its `majority` value must also be x

- (b) both P_i and P_j use the phase king's (P_k) tie-breaker value:
 - as P_k is non-malicious it must have sent the same tie-breaker value to both P_i, P_j
- (c) P_i uses its majority value as the new estimate and P_j uses the phase king's tie-breaker as the new estimate:
 - assume P_i 's majority value is x
 - since P_i uses its majority value x then $w_i > \frac{N}{2} + f$,
 - and of these voters, at least $\frac{N}{2}$ are non-malicious
 - this implies that P_k must also have received at least $\frac{N}{2}$ votes for x
 - implying that its majority value that it sends as tie-breaker must also be x
- for all three possibilities, any two non-malicious processes P_i and P_j agree on the consensus estimate at the end of phase k , where the phase king P_k is non-malicious.

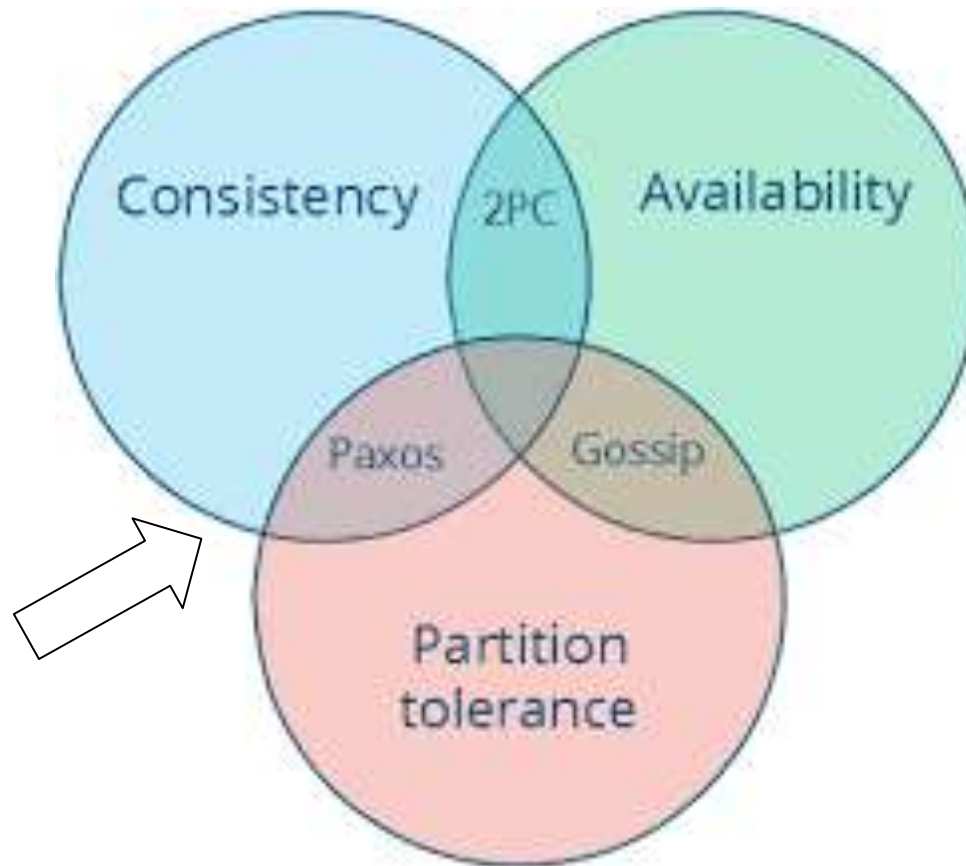
- 3) All non-malicious processes have the same consensus estimate x at the start of phase $k+1$
- and they continue to have the same estimate at the end of phase $k+1$
 - this is self-evident because we have $N > 4f$
 - and each non-malicious process receives at least $N-f > \frac{N}{2} + f$ votes for x from the other non-malicious processes in the first step of phase $k+1$
 - hence, all the non-malicious processes retain their estimate v of the consensus value as x at the end of phase $k+1$

The same logic holds for all subsequent phases.

Hence, the consensus value is correct.



CAP IMPOSSIBILITY



REPLAY



BIBLIOGRAPHY

- [1]  M. J. Fischer, N. A. Lynch, M. S. Paterson, “Impossibility of distributed consensus with one faulty process”, *Journal of the ACM* no. 32, 1985, pp.374-382.
- [2]  L. Lamport, “Paxos made simple”, *ACM SIGACT News* no. 32, vol. 4, 2001, pp.51-58.
- [3]  J. Kirsch, Y. Amir. “Paxos for System Builders”. Technical Report CNDS-2008-2, John Hopkins University, 2008.
- [4]  Ongaro, Diego and John Ousterhout. 2014. “In Search of an Understandable Consensus Algorithm,” *Proc. USENIX ATC’14*, 2014, pp. 305-320.
- [5]  W. Fokkink, “Distributed Algorithms. An Intuitive Approach”, MIT Press, 2013, ch.12-13.
- [6]  Piotr Berman, J.A. Garay, K.J. Perry, "Towards Optimal Distributed Consensus," *Proc. 30th FOCS*, 1989, pp. 410-415
- [7]  R. Guerraoui, M. Hurfin, A. Mostefaoui, R. Oliveira, M. Raynal, A. Schiper, “Consensus in Asynchronous Distributed Systems: A Concise Guided Tour”, *Advances in Distributed Systems: From Algorithms to Systems*, Springer, 2000, pp.33-47.